A - Echolocation

Author: Vincent Trang

Solution

Iterate through the list of Ws. The first W value can only have had contributions from pings at one time, so we can calculate that and subtract it off. Then, we can proceed with the new first value, repeating this process until either we get stuck (impossible) or W is all zeros (possible).

This is best shown by an example. We'll test W = [2, 5, 7, 4, 2], A = [2, 1, 1] with this process:

W	2	5	7	4	2	Initial value of W
-A	-2	-1	-1			Subtract off 1 ping at time t=0
W	0	4	6	4	2	Remaining signal not yet accounted for
-2A		-4	-2	-2		Subtract off 2 pings at time t=1
W	0	0	4	2	2	Remaining signal not yet accounted for
-2A			-4	-2	-2	Subtract off 2 pings at time t=2
W	0	0	0	0	0	All signal is accounted for!

In this case, we found that the recording could be produced with one t=0 ping, two t=1 pings, and two t=2 pings, so the sequence is possible.

There are three ways this process can get stuck. The first occurs if we calculate a non-integer number of pings:

W	2	6	7	4	2	Initial value of W (another example)
-A	-2	-1	-1			Subtract off 1 ping at time t=0
W	0	5	6	4	2	Remaining signal not yet accounted for
-?		?	?	?		Stuck!

Here, the amount of pings we need to subtract off at time t=1 would be calculated as 5/2, which is not an integer. Therefore, we can deduce that the given sequence is impossible.

This process can also get stuck if it encounters a negative number:

W	6	1	6	1	2	Initial value of W (another example)
-3A	-6	-3	-3			Subtract off 3 pings at time t=0
W	0	-2	3	1	2	Negative number!

Any negative number that occurs at any time in the array W should result in a verdict of impossible.

Finally, there can be remaining pings at the end that cannot be accounted for:

W	2	5	7	4	4	Initial value of W (example)
-A	-2	-1	-1			Subtract off 1 ping at time t=0
W	0	4	6	4	4	Remaining signal not yet accounted for
-2A		-4	-2	-2		Subtract off 2 pings at time t=1
W	0	0	4	2	4	Remaining signal not yet accounted for
-2A			-4	-2	-2	Subtract off 2 pings at time t=2
W	0	0	0	0	2	Too few values to remove!

If there are fewer remaining values after the first nonzero element of W than the length of A, this also makes it impossible.

Potential Pitfalls

The most common error was not accounting for the last case, where all subtractions succeed but there were remaining pings unaccounted for. This passed all samples but failed some hidden test cases.

There was also an error that resulted in Time Limit Exceeded: if, when subtracting off pings, you either iterated through all of W or made a copy of W, this would degrade your performance from $O(Tk)$ to $O(T^2)$, which was too slow.

Code (Python)

```
k = int(input())
a = list(map(int, input().split()))
T = int(input())
W = list(map(int, input().split()))
# example: a = 10 3  1
#          W = 20 36 11 3

def main():
    for i in range(len(W)-len(a)+1):
        div1 = W[i]
        div2 = a[0]
        quot = div1 // div2
        for aidx, aval in enumerate(a):
            W[i + aidx] -= aval * quot
            if W[i + aidx] < 0:
                print("NO")
                return
    if any(W):
        print("NO")
    else:
        print("YES")

main()
```

B - Battleship

Author: Vincent Trang

Solution

From each of the Admiral's initial hits, try all four directions; at most three of them can be wrong. Keep a running count of how many hits you've made, and stop once you've made l-1 hits, where l is the ship length. This guarantees at most 3 misses per ship.

Potential Pitfalls

Implementation was the main challenge of this problem. There were lots of loops and potential off-by-one errors to keep track of.

Code (Python)

```
n, k, l = map(int, (input() for _ in range(3)))
grid = [input() for _ in range(n)]
admiral_hits = []
for row in range(n):
    for col in range(n):
        if grid[row][col] == "#":
            admiral_hits.append((row, col))

def query(row, col):
    print("?", row+1, col+1)
    new_grid = [input() for _ in range(n)]
    return new_grid[row][col]

directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
def process(row, col):
    hits_left = l-1
    for (dx, dy) in directions:
        x, y = row, col
        while hits_left:
            x += dx
            y += dy
            if not (0 <= x < n and 0 <= y < n):
                break
            verdict = query(x, y)
            if verdict == ".":
                break
            else:
                hits_left -= 1

for (row, col) in admiral_hits:
    process(row, col)

print("!")
```

C - Mariana Trench

Author: Vincent Trang

Solution

Since N could be up to 10^{17}, any solution which attempted to compute the entire represented string would exceed the time limit. The key is to build a *parse tree*, which will allow us to unroll only the parts of the string that actually matter. An example is given below for the string expression `b+(((a+b)2+a)3)2`:

<table>
<tr><td colspan="3">b+(((a+b)2+a)3)2</td></tr>
<tr><td>*1</td><td colspan="2">*2</td></tr>
<tr><td>b</td><td colspan="2">((a+b)2+a)3</td></tr>
<tr><td></td><td colspan="2">*3</td></tr>
<tr><td></td><td colspan="2">(a+b)2+a</td></tr>
<tr><td></td><td>*2</td><td>*1</td></tr>
<tr><td></td><td>a+b</td><td>a</td></tr>
<tr><td></td><td>*1</td><td>*1</td></tr>
<tr><td></td><td>a</td><td>b</td></tr>
</table>

Here, the string (sub)-expressions are the nodes of our tree, and the counts (`*1, *2, *3`) are labeled edges. Once we have this tree, we can recursively calculate the length of each unrolled expression at each node based on the lengths of its children, then traverse the tree using these lengths to find the N-th character.

It remains to figure out how to generate and store the tree efficiently:

- Storing the entire string expression at each node is inefficient. A way around this is to store just the index of the sub-expression's first character in the original string.
- To generate the parse tree efficiently, we iterate through the string and use a stack.

Here is the process to generate the parse tree:

- Initialize a stack containing a single element 0, and an initially empty adjacency list with size one greater than the length of the input string.
- When you encounter a `(`, push the current 1-based index onto the stack: that is, whatever the index of the `(` is.
- When you encounter a `)` followed by a digit `d`, pop the top element *x* off the stack, and find the new top of the stack *y*. Add an edge from *y* to *x* with label `d`.

- When you encounter a single character at index *i*, find the top of the stack *x*, and add an edge from *x* to *i*. This edge should be labeled with the integer `d` if the single character is followed by a digit `d`; otherwise the label should be 1.
- Any + symbols can be skipped.

Once the parse tree has been generated, all unrolled lengths can be calculated as follows:

- Any leaf node has length 1.
- The length of a non-leaf node is the sum over all its children of the quantity (child length) × (edge label).

Finally, to get the N-th character at a node:

- Leaf nodes are straightforward: N should be 1.
- At a non-leaf node, iterate through its children in order until the running total of (child length) × (edge label) exceeds N. At that point, we compute the total length of the unrolled strings before that child (call this quantity R), and we return the character of the child node at index (N-R) % (child length).

Potential Pitfalls

The most common pitfall was trying to compute the entire represented string. With N up to 10^{17}, this is all but guaranteed to exceed the time limit.

There are also many ways to accidentally go quadratic, such as not precomputing a parse tree using a stack and spending O(n) time at each recursive call to determine the substrings that it splits into, or storing the entire string expression at each node. This would also exceed the time limit.

Finally, in Python, the recursion depth is deep enough that it will lead to a stack overflow. Working Python implementations therefore will implement a stack manually, using a list.

Code (Python)

```
def is_letter(ch):
    return 97 <= ord(ch) <= 122

def is_digit(ch):
    return 48 <= ord(ch) <= 57

s = input()
stack = [0]
edges = [[] for i in range(len(s)+1)]

# generate parse tree
idx = 1
while idx <= len(s):
    ch = s[idx-1]
    if ch == "(":
```

```
        stack.append(idx)
        idx += 1
    elif ch == ")":
        d = int(s[idx])
        child = stack.pop()
        parent = stack[-1]
        edges[parent].append((child, d))
        idx += 2
    elif is_letter(ch):
        parent, child = stack[-1], idx
        if idx < len(s) and is_digit(s[idx]):
            d = int(s[idx])
            idx += 2
        else:
            d = 1
            idx += 1
        edges[parent].append((child, d))
    else:
        idx += 1

# get lengths
INF = 10**17+5
lengths = [1 for i in range(len(s)+1)]
visited = [False for i in range(len(s)+1)]
to_compute = [0]
while to_compute:
    top = to_compute[-1]
    if not edges[top]: # leaf, length initialized to 1
        to_compute.pop()
    elif visited[top]:
        lengths[top] = min(INF, sum(lengths[idx]*weight for idx, weight in edges[top]))
        to_compute.pop()
    else:
        visited[top] = True
        for idx, weight in edges[top]:
            to_compute.append(idx)

# get character
N = int(input())-1
node = 0
while edges[node]:
    tot_len = 0
    for child_idx, weight in edges[node]:
        tot_len += lengths[child_idx] * weight
        if tot_len > N:
            prev_len = tot_len - lengths[child_idx] * weight
            break
    N = (N - prev_len) % lengths[child_idx]
    node = child_idx

print(s[node-1])
```

Code (C++)

```
#include <bits/stdc++.h>
#define pb push_back
#define pii pair<int,int>
#define vi vector<int>
#define vl vector<long long>
#define sz(x) int((x).size())

using namespace std;
using ll = long long;

const set<char> digits = {'2', '3', '4', '5', '6', '7', '8', '9'};

void solve(){
    ll inc = 1;
    string s; cin >> s;
    ll n = sz(s);
    vl len(2*n,0), reps(2*n, 1);
    vector<char> data(2*n, ' ');
    vector<vi> chi(2*n, vi());
    stack<int> pos;
    pos.push(0); //root
    ll last = -1;
    for(ll i=0;i<n;i++){ //children get ordered automatically
        if(s[i] == '+') continue;
        if(s[i] == '('){
            chi[pos.top()].pb(inc);
            pos.push(inc++);
        } else if(s[i] == ')'){
            last = pos.top();
            pos.pop();
        } else if(digits.find(s[i]) != digits.end()){
            reps[last] = (s[i] - '0');
        } else{
            last = inc;
            chi[pos.top()].pb(inc);
            data[inc++] = s[i];
        }
    }
    auto dfs_len = [&](auto self, ll x) ->void {
        if(data[x] == ' '){
            for(ll y : chi[x]){
                self(self, y);
                len[x]+=(len[y] * reps[y]);
                if (len[x] >= 100000000000000001) len[x] = 100000000000000001;
            }
        } else{
            len[x] = 1;
        }
    };
    dfs_len(dfs_len, 0);
```

```
    auto dfs_ans = [&](auto self, ll x, ll num) ->void {
        if(data[x] == ' '){
            for(ll y : chi[x]){
                if(len[y] * reps[y] < num){
                    num -= (len[y] * reps[y]);
                } else{
                    num%=len[y];
                    if(num == 0){
                        num = len[y];
                    }
                    self(self, y, num);
                    break;
                }
            }
        } else{
            cout << data[x] << "\n";
        }
    };
    ll N; cin >> N;
    dfs_ans(dfs_ans, 0, N);
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
    return 0;
}
```

D - Airlocks

Author: Vincent Trang

Solution

First we consider a simpler problem:

> *Given a multiset S of k pressure levels, how many arrangements of S into an array A satisfy A[0] = min(S), and can be closed into a valid loop (that is, adjacent elements differ by 1, and the first and last elements differ by 1)?*

Reading such an array from left to right, given a current value, the next value can either go up by 1 (U), or down by 1 (D). Using this, we can convert an array A into a sequence of ups and downs. For example:

> *1232343212 → 1U, 2U, 3D, 2U, 3U, 4D, 3D, 2D, 1U, 2D (last element wraps around)*

Notice that since our array needs to close into a valid loop, every 1U has to be cancelled out by a 2D: each time we increase the pressure from 1, we eventually need to decrease it back to 1 (from 2). This means the total count of 1U equals the total count of 2D, and in general, the total count of (m)U equals that of (m+1)D.

It turns out that, for each number, we can use this to deduce exactly how many times we need to take U or D from there. We can do this greedily: from 1 (in general, the lowest number in S), we can only go U. This forces the count of 1U, and therefore 2D, since they're equal. Using the total number of 2s, this forces the count of 2U, and therefore 3D. Then we use the total number of 3s, and keep going until all the counts have been forced. In this example:

> *2 ones, 4 twos, 3 threes, 1 four → 2x (1U), 2x (2D), 2x (2U), 2x (3D), 1x (3U), 1x (4D)*

Now, we switch our perspective: instead of viewing the entire array at once, we'll view our path from the perspective of each number. In this example:

> *from 1: we went U, then U*
> *from 2: we went U, then U, then D, then D*
> *from 3: we went D, then U, then D*
> *from 4: we went D.*

From this representation, we can uniquely recover the original array A. This is because we must start from the lowest number, and at each step, we know both what number we're at, and the direction we need to go from that number, using the above representation as instructions. In this example:

Currently at 1. Instructions left: {1 = UU, 2 = UUDD, 3 = DUD, 4 = D}. Next: 1U
Currently at 2. Instructions left: {1 = U, 2 = UUDD, 3 = DUD, 4 = D}. Next: 2U
Currently at 3. Instructions left: {1 = U, 2 = UDD, 3 = DUD, 4 = D}. Next: 3D
Currently at 2. Instructions left: {1 = U, 2 = UDD, 3 = UD, 4 = D}. Next: 2U
Currently at 3. Instructions left: {1 = U, 2 = DD, 3 = UD, 4 = D}. Next: 3U
Currently at 4. Instructions left: {1 = U, 2 = DD, 3 = D, 4 = D}. Next: 4D
Currently at 3. Instructions left: {1 = U, 2 = DD, 3 = D}. Next: 3D
Currently at 2. Instructions left: {1 = U, 2 = DD}. Next: 2D
Currently at 1. Instructions left: {1 = U, 2 = D}. Next: 1U
Currently at 2. Instructions left: {2 = D}. Final: 2D
Result: [1, 2, 3, 2, 3, 4, 3, 2, 1, 2]

Now we just need to characterize the representations that arise from valid arrangements A of the multiset S. The characterization ends up being remarkably simple. The conditions are:

(a) Correct number of Us and Ds from each number (we already showed how to calculate this given S)
(b) Final move is D from all numbers, except the lowest number

It's easy to see that these conditions are necessary for the representation to correspond to a valid array. To see that they're sufficient, we need to show that we never get "stuck" at a number from having exhausted all of that number's instructions before we're done:

We show that m's instructions must be exhausted before (m-1)'s, for all m > 2. Note that (m-1)U and (m)D instructions must alternate being used, and there are the same number of them. Therefore, the last (m)D can't be used until the last (m-1)U has been used. After this point, the next time we reach (m-1) uses the last (m)D instruction, exhausting m's instructions. Since (m-1)'s final move is not U, this proves the claim.

In this example, we showed that 4's instructions must be exhausted before 3's, which must be exhausted before 2's. Once the 2s are done, having the correct number of Us and Ds means we used all the 1s as well, and that the final array is in fact a rearrangement of S.

Now the hard part of the problem is done. Now, we just count how many sets of instructions satisfy conditions (a) and (b). For each number (other than the lowest, which can be ignored), if we need *u* Us and *d* Ds, the number of arrangements ending in a D is (*u+d-1* choose *d-1*). We can multiply them all together to answer our simpler problem of arrangements in a line.

Finally, we solve the original problem. If we have a valid arrangement of airlocks in a circle, and choose a random position to unroll it into a line, the probability that this line starts with the lowest number is (*count of the lowest number*) / (*total count*). In our example from before, there were 2 ones, and 10 numbers total, so this probability would be 2/10. Therefore:

(*# of circle arrangements*) = (*# of line arrangements*) / ((*count of lowest number*) / (*total count*)).

Once we have the formula, the implementation is straightforward.

Code (Python)

```python
N = 1000000007
MAXN = 200005

def powmod(a, b):
    if b == 0: return 1
    if b == 1: return a
    if b%2 != 0: return (a*powmod(a, b-1))%N
    c = powmod(a, b//2)
    return (c*c)%N

def modinv(x):
    return powmod(x, N-2)

factorials = [1 for i in range(MAXN)]
for i in range(1, MAXN):
    factorials[i] = (factorials[i-1] * i) % N

inv_factorials = [1 for i in range(MAXN)]
inv_factorials[MAXN-1] = modinv(factorials[MAXN-1])
for j in range(MAXN-2, 0, -1):
    inv_factorials[j] = ((j+1)*inv_factorials[j+1]) % N

def choose(n, k):
    return (factorials[n] * inv_factorials[n-k] * inv_factorials[k]) % N

def main():
    n = int(input())
    a = list(map(int, input().split()))
    assert len(a) == n
    assert 2 <= n <= 200000
    assert all(1 <= i <= 200000 for i in a)
    tot = sum(a)
    first = a[0]

    bridges = [0]*(len(a)-1)
    for i in range(len(a)-1):
        bridges[i] = a[i]
        if a[i] <= 0:
            print(0)
            return
        a[i+1] -= a[i]
        a[i] = 0

    if any(a):
        print(0)
        return

    prod = (tot * modinv(first)) % N
    for a, b in zip(bridges, bridges[1:]):
        prod *= choose(a+b-1, b)
        prod %= N
    print(prod)

main()
```

E - Hydrothermal Vents (Div. 1)

Author: Vincent Trang

Solution

Perform a [breadth-first search] from A to obtain the length of the shortest path from every cell to A. Do the same from B. Finally, for each checkpoint, add its distances from A and B together, and return the smallest such value.

Potential Pitfalls

Given the constraints, your implementation needed to be fairly efficient. Some Python submissions exceeded the time limit just due to high constant factors (e.g. using sets instead of arrays, or storing more info than needed at each BFS state).

Code (Python)

```
r, c = map(int, input().split())
grid = ["#"*(c+2)] + ["#"+input()+"#" for _ in range(r)] + ["#"*(c+2)]
DIRS = ((1,0),(-1,0),(0,1),(0,-1))
INF = 50000000

def find_char(ch):
    for i, row in enumerate(grid):
        if ch in row:
            return i, row.index(ch)

def bfs(sr, sc):
    dists = [[INF + (grid[rrr][ccc] == "#") for ccc in range(c+2)] \
             for rrr in range(r+2)]
    dists[sr][sc] = 0
    to_process = [(sr, sc)]
    ndl = 0
    while ndl < len(to_process):
        rr, cc = to_process[ndl]
        for dr, dc in DIRS:
            nr, nc = rr+dr, cc+dc
            if dists[nr][nc] == INF:
                to_process.append((nr, nc))
                dists[nr][nc] = dists[rr][cc] + 1
```

```
        ndl += 1
    return dists

a_r, a_c = find_char("A")
b_r, b_c = find_char("B")
a_dists = bfs(a_r, a_c)
b_dists = bfs(b_r, b_c)

smallest = INF
for ridx, r in enumerate(grid):
    for cidx, c in enumerate(r):
        if c == "^":
            smallest = min(smallest, a_dists[ridx][cidx] +
b_dists[ridx][cidx])
print(smallest)
```

F - Ariel Collection Management

Author: Shreyas Mayya

Solution

The idea is to declare some data structure (a hashmap, length-5 array, or even just 5 variables) storing the counts of each object. All counts should start at 0. Then, you would loop through the lines of input, updating the count as you go. Finally, use this data structure to find the maximum count.

Potential Pitfalls

None were too noteworthy. As long as you were able to loop through the input, store the counts appropriately, and recover the item from the data structure you used, there wasn't too much trouble.

Code (Python)

```
counts = {}
n = int(input())
for _ in range(n):
  x = input()
  counts[x] = counts.get(x, 0) + 1
best = max(counts.values())
for item in counts:
  if counts[item] == best:
    print(item)
    print(best)
    break
```

G - Maximum Turmoil

Author: Jacob Ezieme

Solution

Given an assignment of ocean currents, the water in every zone will ends up in a cycle (e.g. Zone 1 → Zone 2 → Zone 3 → Zone 1). So, it returns to its original position every *x* days, where *x* is the length of the cycle. Put differently, it will be in its original position whenever the number of days elapsed is a multiple of *x*. Since different cycles can have different lengths, the number of days it takes for all of them to return would be the least common multiple (LCM) of the cycle lengths.

A list of cycle lengths is possible if the sum of lengths does not exceed the number of zones (n). So, this question now becomes: what is the maximum possible LCM of a list of positive integers with a sum at most n?

This problem can be solved via dynamic programming (DP). Compute an array dp[i][p], where each entry stores the maximum possible LCM of a list of positive integers summing to at most i, all of which have prime factorizations consisting of only the first p primes. This way of doing the DP is motivated by the fact that it only ever makes sense to include cycles whose lengths are distinct prime powers (and so their LCM is just their product), so to find dp[i][p], you just need to iterate through the possible lengths of the cycle whose length is a power of the p-th prime, and use dp[i][p-1] to fill in the rest.

Potential Pitfalls

There were many subtly incorrect DPs. For example, it may be tempting to let dp[i] = optimal solution using integers summing to at most i, and compute dp[i] to be the maximum over all possible cycle lengths c of LCM(c, dp[i-c]). However, this will give the wrong answer for some n, as the optimal solution using a cycle of length c might no longer be optimal when that cycle is taken out. For example, the maximum turmoil for n=29 is 2520 with cycle lengths 9, 8, 7, 5, but the maximum turmoil for n=20 is *not* 8×7×5.

Code (C++)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    int n; cin >> n;
    if (n == 1) {
        cout << 1 << endl;
        return;
    }
    vector<int> primes;
    for (int i = 2; i <= 320; i++) {
        bool b = 1;
        for (int d = 2; d * d <= i; d++) {
            if (i % d == 0) b = 0;
        }
        if (b) primes.push_back(i);
    }
    vector<ll> dp(n+1, 1);
    for (int p : primes) {
        for (int i = n; i >= 1; i--) {
            int cur = 1;
            while (cur <= i) {
                dp[i] = max(dp[i], cur * dp[i - cur]);
                cur *= p;
            }
        }
    }
    cout << dp[n] << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    solve();
}
```

H - Treasure Trove

Author: Vincent Trang

Solution

The first key idea was to use a meet-in-the-middle approach. Since n can be at most 40, split the treasure chests into two groups of 20; for each group, compute the (gold, silver) totals for all 2^20 possible subsets (or fewer if n<40). Let these groups of 2^20 subsets be called A and B.

The next key idea was to binary search on the answer. To check whether a ratio of at least a/b is possible, we assign each type of coin a *value*:

- Assign gold coins a value $b-a$
- Assign silver coins a value $-a$

Then, if there are g gold coins and t total coins, their total value is $g(b-a) + (t-g)(-a) = gb-ta$. This is nonnegative if $gb \geq ta$, or equivalently, $g/t \geq a/b$. Therefore, checking whether a/b can be achieved is equivalent to picking two of these subsets for which the combined value of the coins is nonnegative, and has at least the required number of total coins.

The final key idea was to use a two pointers technique. To set this up, we can filter the set A to contain only the (gold, silver) totals not dominated by another (gold, silver) total in A, in that no different (gold, silver) total in A has at least as many total coins and has at least its value. This can be done by sorting A first by total number of coins, then tiebreaking by value (or equivalently, by gold), using the same process as Div. 2 D. Note that this sorting only needs to be done once: even though the value depends on the ratio being tested, tiebreaking by gold instead avoids this problem. We will also follow this process to filter the set B. This allows us to use the two pointers technique on these filtered lists, as they will both be sorted by total coins and reverse-sorted by value.

Initialize a pointer to the start of the filtered set A and the end of the filtered set B. At each step, compute the combined total and combined value of both subsets being pointed to. If the total is insufficient, advance the pointer of A forward. If the value is insufficient, advance the pointer of B backward. Repeat until the total and value are both sufficient, in which case a solution was found, or a pointer gets advanced outside of array bounds, proving impossibilty.

Potential Pitfalls

There's another way to determine whether two subsets can be chosen to satisfy the conditions, which involves another binary search instead of two pointers. However, this introduces an extra log factor (technically linear, since it's log(2^(n/2))), making it too slow.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n, x;
vector<ll> golds, silvers;

vector<ll> get_all_combos(vector<ll>& arr) {
    vector<ll> totals = {0};
    for (ll val : arr) {
        int sz = totals.size();
        for (int i = 0; i < sz; i++)
            totals.push_back(totals[i] + val);
    }
    return totals;
}

bool test_possible(vector<pair<ll,ll>>& totals1,
                   vector<pair<ll,ll>>& totals2,
                   ll numerator, ll denominator) {
    ll gold_worth = denominator - numerator;
    ll silver_worth = -numerator;

    // Build filtered1: increasing worth, increasing total
    vector<pair<ll,ll>> filtered1, filtered2;
    {
        ll best = LLONG_MIN;
        for (auto& [total, gold] : totals1) {
            ll worth = gold * gold_worth + (total - gold) * silver_worth;
            if (worth > best) {
                best = worth;
                filtered1.push_back({total, worth});
            }
        }
    }
    {
        ll best = LLONG_MIN;
        for (auto& [total, gold] : totals2) {
            ll worth = gold * gold_worth + (total - gold) * silver_worth;
            if (worth > best) {
                best = worth;
                filtered2.push_back({total, worth});
            }
        }
    }

    int ndl1 = 0, ndl2 = (int)filtered2.size() - 1;
    while (true) {
        auto [total1, worth1] = filtered1[ndl1];
        auto [total2, worth2] = filtered2[ndl2];
        if (worth1 + worth2 >= 0 && total1 + total2 >= x)
            return true;
        if (worth1 + worth2 < 0)
            ndl1++;
```

```
        else
            ndl2--;
        if (ndl1 == (int)filtered1.size() || ndl2 == -1)
            return false;
    }
}

void compute_totals(ll denominator = 1000000000LL) {
    int half = n / 2;

    vector<ll> g1(golds.begin(), golds.begin() + half);
    vector<ll> s1(silvers.begin(), silvers.begin() + half);
    auto gt1 = get_all_combos(g1);
    auto st1 = get_all_combos(s1);
    vector<pair<ll,ll>> totals1;
    for (int i = 0; i < (int)gt1.size(); i++)
        totals1.push_back({gt1[i] + st1[i], gt1[i]});
    sort(totals1.begin(), totals1.end(), greater<>());

    vector<ll> g2(golds.begin() + half, golds.end());
    vector<ll> s2(silvers.begin() + half, silvers.end());
    auto gt2 = get_all_combos(g2);
    auto st2 = get_all_combos(s2);
    vector<pair<ll,ll>> totals2;
    for (int i = 0; i < (int)gt2.size(); i++)
        totals2.push_back({gt2[i] + st2[i], gt2[i]});
    sort(totals2.begin(), totals2.end(), greater<>());

    ll left = 0, right = denominator;
    while (right - left > 1) {
        ll mid = (left + right) / 2;
        if (test_possible(totals1, totals2, mid, denominator))
            left = mid;
        else
            right = mid;
    }

    cout << fixed << setprecision(9) << (double)left / denominator << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> x;
    golds.resize(n);
    silvers.resize(n);
    for (auto& g : golds) cin >> g;
    for (auto& s : silvers) cin >> s;

    compute_totals();
    return 0;
}
```

I - Pronounceable Names

Author: Vincent Trang

Solution

Split the string into blocks of consonants and vowels. An n-letter block requires exactly ⌊n/3⌋ replacements; add this up among all the blocks.

To prove that ⌊n/3⌋ replacements suffice, simply flip the 3rd, 6th, 9th, 12th, … letters in each block (consonants ↔ vowels). Exception: if n is a multiple of 3, flip the 2nd, 5th, 8th, 11th, … letters instead.

To prove that ⌊n/3⌋ replacements are necessary, split the n-letter block into ⌊n/3⌋ disjoint 3-letter blocks. Clearly, at least one replacement is necessary per 3-letter block, since it starts out as all consonants/vowels.

Potential Pitfalls

The main pitfalls were implementation-based: when iterating through the string, the run-length needs to be reset to 1 (not 0) if the consonant/vowel type changes. Additionally, if you also reset when the run-length reached 3, you need to reset to 0 (not 1) in this case.

Code (Python)

```
is_vowel = list(map(lambda x: x in "AEIOU", input()))
streaks = [1]
for curr, prev in zip(is_vowel[1:], is_vowel):
    if curr == prev:
        streaks[-1] += 1
    else:
        streaks.append(1)
print(sum(i//3 for i in streaks))
```

J - Food Not-Chain (Div. 1)

Author: Vincent Trang

Solution

First, use n-1 queries in a "reverse knockout" tournament: query species 1 and 2 first, and then at each step, test the loser of the previous query against the next-numbered species. Repeat until all species have been tested. The loser of the final query can be called the *ultimate loser*.

Note that every species other than the ultimate loser beat exactly one species. If the species it beat wasn't the ultimate loser, it must have beat a different species in a later round, and so on, so there are no cycles. Therefore, each species has a chain of wins (i.e. A beats B beats C beats D…) which ends in the ultimate loser.

Now, use n-1 queries to determine one species that the ultimate loser beats, just querying it against every other species. The problem statement guarantees that such a species exists; call this species the *cycler*. From the previous paragraph, there must be a chain of wins from the cycler to the ultimate loser, and since the ultimate loser beats the cycler, this gives us a cycle of wins (i.e. A beats B beats C beats … beats Z beats A).

Finally, if this cycle has length longer than 3, we can use a query to compress it. At each step, if we query A and C, we'll either get "C beats A" (in which case we can report the 3-cycle A B C), or we get "A beats C", which gives us a cycle with one fewer species (i.e. A beats C beats … beats Z beats A). This takes at most n-3 queries to get a cycle of length 3.

Code (Python)

```
import sys

def make_query(a, b):
    print("?", a, b)
    sys.stdout.flush()
    return int(sys.stdin.readline())

def make_answer(a, b, c):
    print("!", a, b, c)
    sys.stdout.flush()

def do_case():
    n = int(input())
    beat = [None for i in range(n+1)]
    prev_loser = 1
    for i in range(2, n+1):
        winner = make_query(prev_loser, i)
        loser = prev_loser + i - winner
        beat[winner] = loser
```

```
        prev_loser = loser

    # prev_loser is the biggest loser. But they must have beat someone. See who.
    loser_of_loser = None
    for i in range(1, n+1):
        if i != prev_loser:
            winner = make_query(prev_loser, i)
            if winner == prev_loser:
                loser_of_loser = i
                break

    if loser_of_loser is None:
        print("Debug")

    # now we have a cycle, loser > cycler > beat[cycler] > beat[beat[cycler]] > ... > loser.
    # abbreviate to 1 > 2 > 3 > 4 > 5 > 6 > 7 > 1 (e.g.).
    cycle = [loser, loser_of_loser]
    while cycle[-1] != loser:
        cycle.append(beat[cycle[-1]])
    while len(cycle) > 3:
        winner = make_query(cycle[-3], cycle[-1])
        if winner == cycle[-1]:
            make_answer(cycle[-3], cycle[-2], cycle[-1])
            return
        else:
            temp = cycle[-1]
            cycle.pop()
            cycle.pop()
            cycle.append(temp)
    make_answer(cycle[-3], cycle[-2], cycle[-1])

t = int(input())
for _ in range(t):
    do_case()
```

K - The Kraken

Author: Vincent Trang

Solution

The simplest solution was to initialize a 2D array of chars, all initialized to dots (.), then going left-to-right through the grid, adding hashes (#) in the correct places (by tracking which row the previous one was, what the current direction of the zigzag is, and updating accordingly). Some teams also solved this problem going row-by-row with modular arithmetic.

Potential Pitfalls

It may have been tempting to commit to a row-by-row approach without realizing that column-by-column was possible (and easier) by storing intermediate results in an array before printing. Otherwise, there were no real pitfalls: every Div. 2 team who attempted this problem solved it on the first try.

Code (Python)

```
width, height = map(int, input().split())
grid = [["." for col in range(width)] for row in range(height)]
curr_row = 0
curr_dir = 1
for curr_col in range(width):
    grid[curr_row][curr_col] = "#"
    if curr_row + curr_dir >= height or curr_row + curr_dir < 0:
        curr_dir *= -1
    curr_row += curr_dir
for i in grid:
    print("".join(i))
```

L - The Leviathan

Author: Vincent Trang

Solution

The main observation was that the reachable squares form a checkerboard pattern on every other row/column. However, there's a special case: if the grid is too small (i.e. on either the horizontal or vertical axis, you're less than 2 spaces from both edges), you can't move. So:

- If you can move (i.e. in both the horizontal and vertical directions, you're at least 2 spaces from the cells on the edge), the conditions are:
  - x_1 and x_2 have same parity
  - y_1 and y_2 have same parity
  - $\lfloor (x_1 + x_2)/2 \rfloor$ and $\lfloor (y_1 + y_2)/2 \rfloor$ have same parity
- If you can't move:
  - You need $x_1 == x_2$ and $y_1 == y_2$.

Potential Pitfalls

The sample cases for this problem were intentionally weak, as this problem was meant to test how carefully you thought about edge cases. Most teams got the first two parity conditions, some got the third, but only a few also found the special cases.

Additionally, in the problem statement, x referred to the vertical axis, while y referred to the horizontal axis, which may have been confusing. The constraints and the sample both pointed to the correct interpretation, however.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    int m, n, x1, x2, y1, y2;
    cin >> m >> n >> x1 >> y1 >> x2 >> y2;
    if ((x1 == x2) && (y1 == y2)) {
        cout << "YES";
    } else if ((m <= 2) || (n <= 2)) {
        cout << "NO";
    } else if (((m == 3) && (x1 == 2)) || ((n == 3) && (y1 == 2))) {
        cout << "NO";
    } else if (((x1-x2)%2 != 0) || ((y1-y2)%2 != 0)) {
        cout << "NO";
    } else if (((x2-x1)+(y2-y1))%4 != 0) {
        cout << "NO";
    } else {
        cout << "YES";
    }
}
```

M - Stacked Coral

Author: Vincent Trang

Solution

This problem can be solved via dynamic programming (DP).

For each (i, j), check if the bottom i elements of the 1st stack and the bottom j elements of the 2nd stack can create the top i+j elements of the target stack. Store this in an array element dp[i][j]. This depends only on dp[i-1][j] (corresponding to a solution where the j-th element of the 2nd stack is placed as the (i+j)-th element from the top of the target stack), and dp[i][j-1] (the analogous correspondence for the 1st stack).

To obtain a valid sequence, either store which stack was used while calculating the dp table, or iteratively recover one using the completed table. The given solution uses the second approach.

Potential Pitfalls

If using the first approach, you cannot store the entire path so far at each DP state, as this will degenerate to O(n^3). Storing just the single next step at each DP state is enough, as you can recover the entire path iteratively at the end.

Code (Python)

```
s1 = input()
s2 = input()
targ = input()
assert 1 <= len(s1) <= 4000
assert 1 <= len(s2) <= 4000
assert len(s1) + len(s2) == len(targ)
assert all(i in "ABCD" for i in s1 + s2 + targ)

dp = [[False for i in range(len(s2)+1)] for j in range(len(s1)+1)]
# dp[i][j] = is it possible to get the last (i+j) chars of targ
# from the first i chars of s1 and the first j chars of s2

dp[0][0] = True

for i in range(len(s1)+1):
    for j in range(len(s2)+1):
        if i or j:
```

```
            dp[i][j] = ((i and dp[i-1][j] and (s1[i-1] == targ[-i-j])) or
(j and dp[i][j-1] and (s2[j-1] == targ[-i-j])))

if not dp[-1][-1]:
    print("IMPOSSIBLE")
else:
    sol = []
    i = len(s1)
    j = len(s2)
    while i or j:
        if i and dp[i-1][j]:
            sol.append("1")
            i -= 1
        else:
            sol.append("2")
            j -= 1
    print("".join(sol))
```